

STOCKPILOT AI

Compte rendu de progression - Jour 1

Session du 3 au 4 août 2026

5 documents de cadrage	7 tables PostgreSQL	5 schémas de données	175 200 snapshots d'inventaire
-------------------------------------	-------------------------------	--------------------------------	---

Objectif de la journée

Passer d'une idée de SaaS à une fondation technique reproductible : cadrage, Git, Docker, PostgreSQL et données synthétiques réalistes.

Dépôt : github.com/Tahamahir/stockpilot-ai

1. Résumé exécutif

Résultat principal. La première journée a permis de transformer StockPilot AI en un projet structuré, versionné et exécutable. Le périmètre du MVP est défini, l'architecture cible est documentée, PostgreSQL fonctionne dans Docker et le modèle relationnel initial est opérationnel.

Fondation Data. Les tables métier centrales ont été créées avec leurs contraintes d'intégrité et leurs index. Un jeu de données minimal a validé les relations SQL, puis un générateur de données synthétiques réalistes a été mis en place pour préparer les futurs pipelines ETL et les modèles de prévision.

Prochaine priorité. Le prochain jalon est l'intégration d'Apache Airflow afin de détecter, contrôler et charger automatiquement les fichiers CSV dans le schéma raw.

État d'avancement à la fin du Jour 1

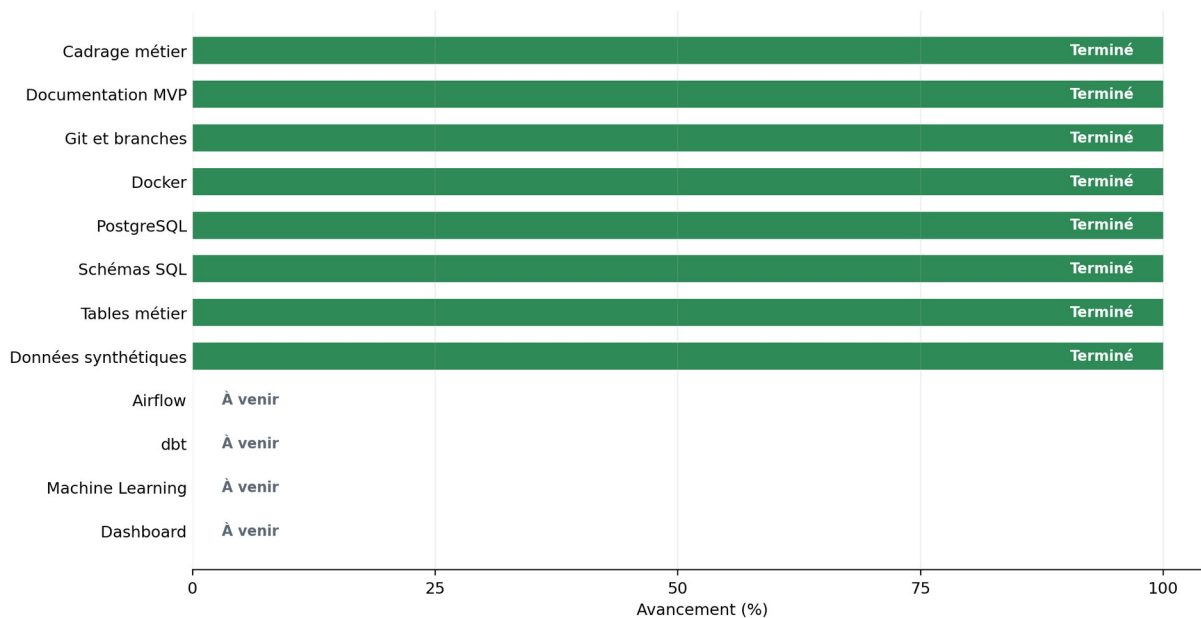


Figure 1 - Avancement global après la première session de travail.

2. Travaux réalisés

2.1 Cadrage métier et produit

- Définition du client cible initial : petites et moyennes entreprises gérant leurs ventes et stocks principalement dans Excel ou CSV.
- Formulation de la question métier centrale : quels produits commander, en quelle quantité et pourquoi ?
- Définition du périmètre du MVP, des fonctionnalités incluses et des éléments reportés après validation.
- Rédaction des user stories pour le gérant, le responsable des achats, le responsable de stock et l'analyste.
- Création du dictionnaire de données et de l'architecture technique cible.

2.2 Fichiers de documentation créés

Fichier	Rôle
---------	------

docs/business_problem.md	Problème métier, client cible et proposition de valeur.
docs/mvp_scope.md	Fonctionnalités incluses, exclusions et critères de réussite.
docs/user_stories.md	Besoins utilisateurs et critères d'acceptation.
docs/data_dictionary.md	Définition initiale des entités et colonnes.
docs/architecture.md	Flux de données, rôle des outils et trajectoire vers le SaaS.

2.3 Gestion du code avec Git

Le dépôt GitHub a été initialisé avec les branches **main**, **develop** et une branche fonctionnelle pour le générateur de données. Les commits sont organisés par objectif, ce qui prépare les futurs pull requests et la CI/CD.

Repository : <https://github.com/Tahamahir/stockpilot-ai.git>
Branches : main, develop, feature/synthetic-data-generator
État : working tree clean et branches synchronisées

3. Architecture technique retenue

L'architecture cible sépare clairement ingestion, transformation, modélisation, exposition des résultats et interface utilisateur. Cette séparation rend le système testable, remplaçable par module et évolutif vers une offre multi-entreprises.

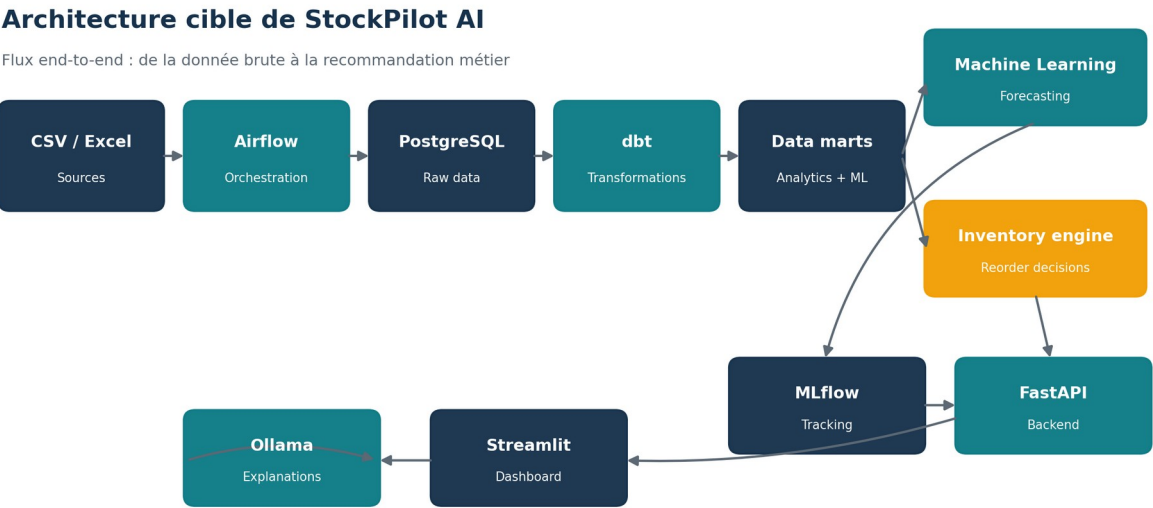


Figure 2 - Architecture end-to-end prévue pour le MVP.

Composant	Responsabilité principale
-----------	---------------------------

Apache Airflow	Orchestrer les imports, contrôles, transformations et entraînements.
PostgreSQL	Stocker les données raw, staging, analytics et ML.
dbt	Nettoyer, tester, documenter et produire les data marts.
MLflow	Tracer les expériences et versionner les modèles.
FastAPI	Exposer les prévisions, alertes et recommandations.
Streamlit	Présenter les indicateurs et décisions métier.
Ollama	Expliquer les résultats sans inventer les calculs.

4. Environnement Docker et PostgreSQL

4.1 Validation de Docker

Docker Desktop a d'abord rencontré un blocage du moteur Linux. Le problème a été résolu en redémarrant correctement Docker/WSL, puis le service PostgreSQL a été lancé avec succès.

```
Docker version      : 28.3.2
Docker Compose version: v2.39.1-desktop.1
Image PostgreSQL    : postgres:17-alpine
Port local          : 5432
État du conteneur   : Up (healthy)
```

4.2 Schémas PostgreSQL créés

La base stockpilot est organisée en cinq couches pour éviter de mélanger les données brutes, les transformations et les résultats analytiques :

raw	staging	intermediate	analytics	ml
-----	---------	--------------	-----------	----

4.3 Tables métier opérationnelles

Les tables companies, stores, suppliers, products, sales et inventory sont créées avec des clés étrangères, des contraintes de contrôle, des index et une logique tenant_id préparant l'évolution vers un SaaS multi-entreprises.

Modèle relationnel initial

Les clés composées tenant_id + identifiant préparent l'isolation multi-entreprises.

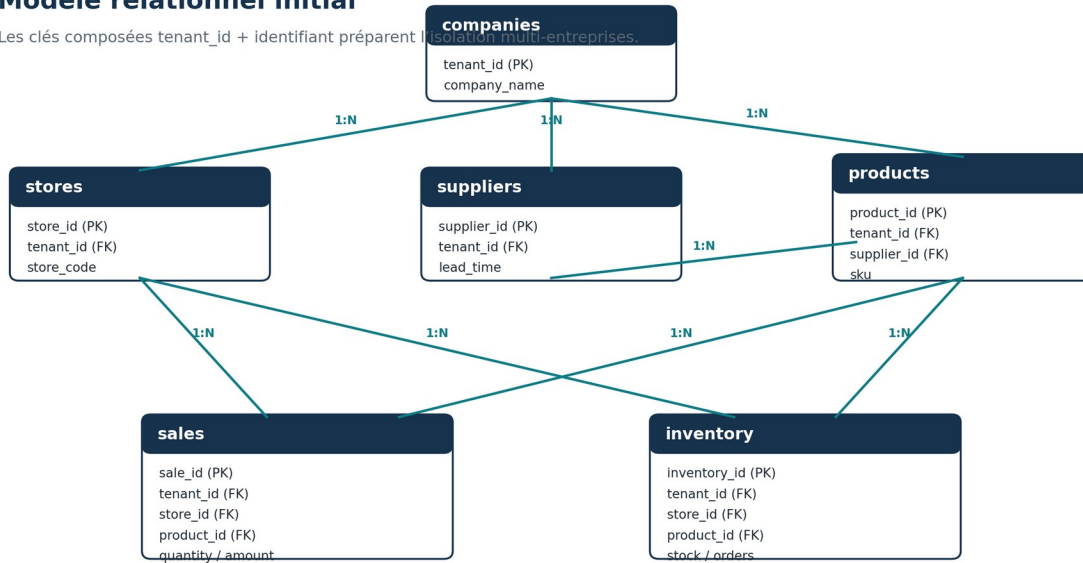


Figure 3 - Modèle relationnel initial de la couche raw.

```

raw.companies
raw.stores
raw.suppliers
raw.products
raw.sales
raw.inventory
raw.pipeline_test
  
```

5. Données de démonstration

5.1 Deux niveaux de données

Données minimales SQL. Elles ont servi à vérifier les clés étrangères, les contraintes et les premières requêtes métier, notamment le calcul de la position de stock.

Données synthétiques réalistes. Un générateur Python reproductible a ensuite été créé pour simuler des volumes suffisants pour Airflow, dbt, le Machine Learning et la démonstration du produit.

Configuration du dataset synthétique de démonstration



Figure 4 - Configuration officielle du dataset synthétique.

5.2 Comportements simulés

- saisonnalité hebdomadaire et annuelle ;
- tendance croissante ou décroissante selon le produit ;
- effet des promotions et des remises ;
- produits à forte rotation, faible rotation et demande intermittente ;
- ruptures de stock, demande non satisfaite et backorders ;
- délais fournisseurs et retards de livraison ;
- réapprovisionnement par quantité minimale et taille de colis ;
- fichier invalide séparé pour tester les futurs contrôles ETL.

5.3 Limite méthodologique

Point de vigilance : les données synthétiques sont adaptées au développement et à la démonstration, mais elles ne prouvent pas à elles seules la performance commerciale. Une version vendable devra être validée par backtesting sur les données d'une entreprise pilote.

6. Validations techniques effectuées

Élément	Statut	Preuve / résultat
Git	VALIDÉ	Dépôt initialisé, branches distantes créées et état propre.
Docker	VALIDÉ	Client et moteur opérationnels après résolution du blocage WSL.
PostgreSQL	VALIDÉ	Conteneur actif et healthcheck au statut healthy.
SQL	VALIDÉ	Schémas, tables, contraintes, clés étrangères et index créés.
Données	VALIDÉ	Données de test insérées et requêtes métier validées.
Python	VALIDÉ	Générateur synthétique compilé et exécuté avec seed

reproductible.

7. Incident rencontré et résolution

Incident. Docker Compose ne pouvait pas télécharger l'image PostgreSQL, car le canal dockerDesktopLinuxEngine était indisponible. Docker Desktop restait bloqué au démarrage pendant plus de trente minutes.

Diagnostic. Le client Docker était installé, mais le moteur Linux n'était pas actif.

Résolution. Arrêt de la commande, redémarrage de Docker Desktop/WSL, puis nouvelle exécution de Docker Compose. Le conteneur PostgreSQL a ensuite atteint le statut healthy.

8. Livrables disponibles à la fin du Jour 1

- Repository GitHub structuré avec main, develop et branche fonctionnelle.
- Documentation complète du problème métier et du MVP.
- Architecture technique cible documentée.
- docker-compose.yml avec PostgreSQL 17.
- Scripts SQL d'initialisation, création des tables et données de démonstration.
- Modèle relationnel multi-tenant initial.
- Générateur Python de données commerciales synthétiques.
- Configuration reproductible : 730 jours, 120 produits, 2 magasins et 8 fournisseurs.
- Jeu de fichiers volontairement invalides pour les futurs tests de qualité.

9. Prochaines étapes - Jour 2

1	Ajouter Apache Airflow au Docker Compose.
2	Créer la connexion Airflow vers PostgreSQL.
3	Développer un DAG de détection et validation des fichiers CSV.
4	Charger les enregistrements valides dans raw et isoler les lignes rejetées.
5	Journaliser les erreurs dans raw.rejected_records.
6	Ajouter les premiers tests automatisés du pipeline d'ingestion.

Conclusion : le projet dispose désormais d'une base saine pour démarrer la Data Engineering. Le cœur du Jour 2 sera l'automatisation de l'ingestion, sans encore introduire dbt ni les modèles de prévision.

Note de traçabilité : ce compte rendu synthétise les commandes, scripts et validations de la session. Aucune performance Machine Learning n'a encore été mesurée.